

scripts/hooks/guard-forbidden-commands.sh — User Guide

Last verified: 2026-06-02 (anti-forgetting enforcement cycle)
Inheritance: §6.X / §6.V / §6.AG (emulators), §6.T.3 (force-push/--no-verify), §6.U (no-sudo), Host Machine Stability Directive; HelixConstitution §11.4.18 (script docs) **Classification:** universal

Overview

A Claude Code PreToolUse hook that mechanically BLOCKS forbidden/gated Bash commands at the tool-call boundary — so the constraints stop depending on an agent (or orchestrator) remembering them. It is the "floor": even if a subagent prompt omits a rule, the command is still refused.

Contract (Claude Code PreToolUse hook)

- Reads the tool invocation as JSON on **stdin**. For `tool_name == "Bash"` the command string is at `.tool_input.command`.
- **Exit 0** → allow (Claude proceeds).
- **Exit 2** → BLOCK; the `stderr` text is fed back to Claude as the refusal reason.
- Non-Bash tools and empty commands pass through (exit 0).
- Uses `jq` when present; otherwise a built-in `awk` JSON-string extractor (no hard dependency).

Blocked command classes

Class	Clause	Examples blocked
Raw host-direct emulator / device	§6.X / §6.V / §6.AG	<code>emulator -avd ..., \$ANDROID_HOME/emulator/emulator ...</code> <code>adb install ..., adb -s ... install ...</code> <code>am instrument ...</code>

Class	Clause	Examples blocked
Force-push / verification-bypass	§6.T.3	git push --force, git push -f, git push --force-with-lease, --no-verify, --no-gpg-sign
Privilege escalation	§6.U	sudo ..., su, su - ..., su -l ...
Host power	Host Machine Stability Directive	systemctl suspend/hibernate/poweroff/reboot/halt/..., loginctl ..., pm-suspend, shutdown ...

Read-only / ordinary commands (adb logcat, ./gradlew test, git status, git push origin <branch>, scripts/run-challenge-matrix.sh ...) pass through.

Escape hatch (documented exceptions only)

A command containing the literal marker `# guardrails:allow <reason>` is **WARNED** (printed to stderr) but **NOT blocked**. The `<reason>` text is mandatory so the exception is self-documenting in the transcript. Use it **ONLY** for operator-approved actions (e.g. an authorized mirror-reconcile force-push, or a dev-iteration host-direct emulator run that is explicitly not gate evidence).

```
git push --force origin master # guardrails:allow operator
approved mirror reconcile 2026-06-02
```

The host-power class is NOT overridable — the marker is ignored for systemctl/loginctl/pm-*/shutdown. There is no in-band reason that overrides "do not power off the operator's machine".

Wiring

Configured in `.claude/settings.json`:

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Bash",
        "hooks": [ { "type": "command",
                     "command": "${CLAUDE_PROJECT_DIR}/scripts/
hooks/guard-forbidden-commands.sh" } ] }
    ]
  }
}
```

```
}  
}
```

The hook takes effect when Claude Code (re)loads project settings — it does NOT apply mid-session in an already-running session.

The orchestrator should merge `.claude/settings.json` at a safe time (after any in-flight emulator run).

Usage / manual testing

```
printf '%s' '{"tool_name":"Bash","tool_input":{"command":"adb  
install x.apk"}}' \  
| bash scripts/hooks/guard-forbidden-commands.sh ; echo  
"exit=$?" # → exit=2
```

Hermetic test

`tests/hooks/guard_forbidden_commands_test.sh` — 28 cases asserting exit codes for every blocked class, the allowed commands, the escape hatch, the non-overridable host-power class, and non-Bash pass-through. Run:

```
bash tests/hooks/guard_forbidden_commands_test.sh
```

Limitations

The hook is a syntactic command-pattern matcher; it cannot enforce semantic rules (anti-bluff intent, resource caps, evidence honesty). Those remain the agent's responsibility per `docs/AGENT_GUARDRAILS.md` (the hook is the floor, the preamble is the ceiling). An adversary could obfuscate a command to evade the regex; the hook targets honest-mistake / forgotten-rule cases, not a hostile actor with shell access.